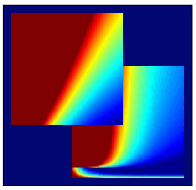
Om Sanan

CS/CNS/EE 156a Learning Systems
Caltech - Fall 2025
https://caltech.instructure.com/courses/8882
(Learning From Data campus version)



Homework # 8

Due Monday, November 24, 2025, at 2:00 PM PST

*Definitions and notation follow the lectures. All questions have multiple-choice answers (***[a], [b], [c], ...***). Collaboration is allowed but without discussing selected or excluded choices. Your solutions must be based on your own work. See the initial "***Course Description and Policies***" handout for important details about collaboration, open book, and chatGPT policies.*

Note about the homework

- Answer each question by deriving the answer (carries 6 points) then selecting from the multiple-choice answers (carries 4 points). You can select 1 or 2 of the multiple-choice answers for each question, but you will get 4 or 2 points, respectively, for a correct answer. See the initial "**Course Description and Policies**" handout for important details.
- The problems range from easy to difficult, and from practical to theoretical. Some problems require running a full experiment to arrive at the answer.
- The answer may not be obvious or numerically close to one of the choices, but one (and only one) choice will be correct if you follow the instructions precisely in each problem. You are encouraged to explore the problem further by experimenting with variations on these instructions, for the learning benefit.
- You are encouraged to take part in the Piazza discussion forum. Please make sure you don't discuss specific answers, or specific excluded answers, before the homework is due.

© 2012-2025 Yaser Abu-Mostafa. All rights reserved. No redistribution in any format. No translation or derivative products without written permission.

- **Primal versus Dual Problem**

1. Recall that N is the size of the data set and d is the dimensionality of the input space. The original formulation of the hard-margin SVM problem (minimize $\frac{1}{2}\mathbf{w}^{\mathrm{T}}\mathbf{w}$ subject to the inequality constraints), without going through the Lagrangian dual problem, is

w is a vector in the input space of dimension d and b is scalar. therefore d+1 variables.

[**a**] a quadratic programming problem with N variables

[**b**] a quadratic programming problem with $N+1$ variables

[**c**] a quadratic programming problem with d variables

[**d**] a quadratic programming problem with $d+1$ variables

[**e**] not a quadratic programming problem

The goal of the SVM is to find the optimal hyperplane defined by the weights w and bias b.

Minimize: $1/2 * w^T * w$ -> quadratic function of variables; subject to $y_n(w^T x_n + b) \geq 1$.

Notice: The following problems deal with a real-life data set. In addition, the computational packages you use may employ different heuristics and require different tweaks. This is a typical situation that a Machine Learning practitioner faces. There are uncertainties, and the answers may or may not match our expectations. Although this situation is not as 'sanitized' as other homework problems, it is important to go through it as part of the learning experience.

SVM with Soft Margins

In the rest of the problems of this homework set, we apply soft-margin SVM to handwritten digits from the processed US Postal Service Zip Code data set. Download the data (extracted features of intensity and symmetry) for training and testing:

http://www.amlbook.com/data/zip/features.train

http://www.amlbook.com/data/zip/features.test

(the format of each row is: **digit intensity symmetry**). We will train two types of binary classifiers; one-versus-one (one digit is class $+1$ and another digit is class -1, with the rest of the digits disregarded), and one-versus-all (one digit is class $+1$ and the rest of the digits are class -1).

The data set has thousands of points, and some quadratic programming packages cannot handle this size. You may need stronger SVM packages such as **fitcsvm** in MatLab or the free **libsvm** (http://www.csie.ntu.edu.tw/~cjlin/libsvm/).

Implement SVM with soft margin on the above zip-code data set by solving

$$\begin{aligned} \min_{\boldsymbol{\alpha}} \quad & \frac{1}{2}\sum_{n=1}^{N}\sum_{m=1}^{N} \alpha_n \alpha_m y_n y_m K(\mathbf{x}_n, \mathbf{x}_m) - \sum_{n=1}^{N} \alpha_n \\ \text{s.t.} \quad & \sum_{n=1}^{N} y_n \alpha_n = 0 \\ & 0 \le \alpha_n \le C \quad n = 1, \cdots, N \end{aligned}$$

When evaluating E_{in} and E_{out} of the resulting classifier, use binary classification error. E_{out} is estimated using the test set.

Practical remarks:

(i) For the purpose of this homework, do not scale the data when you use libsvm or other packages, otherwise you may inadvertently change the (effective) kernel and get different results.

(ii) In some packages, you need to specify double precision.

(iii) In 10-fold cross validation, if the data size is not a multiple of 10, the sizes of the 10 subsets may be off by 1 data point.

(iv) Some packages have software parameters whose values affect the outcome. ML practitioners have to deal with this kind of added uncertainty.

- **Polynomial Kernels**

Consider the polynomial kernel $K(\mathbf{x}_n, \mathbf{x}_m) = (1 + \mathbf{x}_n^{\mathrm{T}} \mathbf{x}_m)^Q$, where Q is the degree of the polynomial.

2. With $C = 0.01$ and $Q = 2$, which of the following classifiers has the **highest** E_{in}?

[**a**] 0 versus all

[**b**] 2 versus all

[**c**] 4 versus all

[**d**] 6 versus all

[**e**] 8 versus all

3. With $C = 0.01$ and $Q = 2$, which of the following classifiers has the **lowest** E_{in}?

[**a**] 1 versus all

[**b**] 3 versus all

[c] 5 versus all

[d] 7 versus all

[e] 9 versus all

4. Comparing the two selected classifiers from Problems 2 and 3, which of the following values is the closest to the difference between the number of support vectors of these two classifiers?

[a] 600

[b] 1200

[c] 1800

[d] 2400

[e] 3000

5. Consider the 1 versus 5 classifier with $Q = 2$ and $C \in \{0.001, 0.01, 0.1, 1\}$. Which of the following statements is correct? Going up or down means strictly so.

[a] The number of support vectors goes down when C goes up.

[b] The number of support vectors goes up when C goes up.

[c] E_{out} goes down when C goes up.

[d] Maximum C achieves the lowest E_{in}.

[e] None of the above

6. In the 1 versus 5 classifier, comparing $Q = 2$ with $Q = 5$, which of the following statements is correct?

[a] When $C = 0.0001$, E_{in} is higher at $Q = 5$.

[b] When $C = 0.001$, the number of support vectors is lower at $Q = 5$.

[c] When $C = 0.01$, E_{in} is higher at $Q = 5$.

[d] When $C = 1$, E_{out} is lower at $Q = 5$.

[e] None of the above

• Cross Validation

In the next two problems, we will experiment with 10-fold cross validation for the polynomial kernel. Because E_{cv} is a random variable that depends on the random partition of the data, we will try 100 runs with different partitions and base our answer on how many runs lead to a particular choice.

7. Consider the 1 versus 5 classifier with $Q = 2$. We use E_{cv} to select $C \in \{0.0001, 0.001, 0.01, 0.1, 1\}$. If there is a tie in E_{cv}, select the smaller C. Within the 100 random runs, which of the following statements is correct?

[a] $C = 0.0001$ is selected most often.

[b] $C = 0.001$ is selected most often.

[c] $C = 0.01$ is selected most often.

[d] $C = 0.1$ is selected most often.

[e] $C = 1$ is selected most often.

8. Again, consider the 1 versus 5 classifier with $Q = 2$. For the overall winning selection in the previous problem, the average value of E_{cv} over the 100 runs is closest to

[a] 0.001

[b] 0.003

[c] 0.005

[d] 0.007

[e] 0.009

- **RBF Kernel**

Consider the radial basis function (RBF) kernel $K(\mathbf{x}_n, \mathbf{x}_m) = \exp\left(-||\mathbf{x}_n - \mathbf{x}_m||^2\right)$ in the soft-margin SVM approach. Focus on the 1 versus 5 classifier.

9. Which of the following values of C results in the lowest E_{in}?

[a] $C = 0.01$

[b] $C = 1$

[c] $C = 100$

[d] $C = 10^4$

[e] $C = 10^6$

10. Which of the following values of C results in the lowest E_{out}?

[a] $C = 0.01$

[b] $C = 1$

[c] $C = 100$

[d] $C = 10^4$

[e] $C = 10^6$

Problems 2-10

```
import numpy as np
from sklearn.svm import SVC


def load_data(filename):
    data = np.loadtxt(filename)
    digits = data[:, 0].astype(int)
    X = data[:, 1:]
    return X, digits

def one_vs_all(digits, target):
    """
    convert to binary labels for 1vall classification
    """
    y = np.where(digits == target, 1, -1)
    return y

def one_vs_one(X, digits, pos_digit, neg_digit):
    """
    filter data for 1v1 classification
    """
    mask = (digits == pos_digit) | (digits == neg_digit)
    X_filtered = X[mask]
    y_filtered = np.where(digits[mask] == pos_digit, 1, -1)
    return X_filtered, y_filtered

def train_svm_poly(X, y, C, Q):
    """
    train SVM with kernel K(x,y) = (1 + x^T y)^Q
    sklearn uses: (gamma * x^T y + coef0)^degree
    so we set: gamma=1, coef0=1, degree=Q
    """
    svm = SVC(kernel='poly', degree=Q, gamma=1, coef0=1, C=C)
    svm.fit(X, y)
    return svm

def calculate_classification_error(y_true, y_pred):
    """
    calculates binary classification error
    """
    misclassified_count = np.sum(y_true != y_pred)
    error_rate = misclassified_count / len(y_true)
    return error_rate

def get_num_support_vectors(svm):
    """
```

```
    returns the number of support vectors
    """
    return len(svm.support_)

def cross_validation_error(X, y, C, Q, n_folds=10):
    """
    compute 10 fold cross validation error
    """
    N = len(y)
    indices = np.arange(N)

    # shuffle indices
    np.random.shuffle(indices)

    # split into n_folds
    fold_sizes = np.full(n_folds, N // n_folds)
    fold_sizes[:N % n_folds] += 1

    errors = []
    current = 0

    for fold_size in fold_sizes:
        # validation indices for this fold
        val_indices = indices[current:current + fold_size]
        # training indices = everything else
        train_indices = np.concatenate([indices[:current],
indices[current + fold_size:]])

        X_train_fold = X[train_indices]
        y_train_fold = y[train_indices]
        X_val_fold = X[val_indices]
        y_val_fold = y[val_indices]

        # train and predict
        svm = train_svm_poly(X_train_fold, y_train_fold, C, Q)
        y_pred = svm.predict(X_val_fold)

        # compute error for this fold
        fold_error = calculate_classification_error(y_val_fold,
y_pred)
        errors.append(fold_error)

        current += fold_size

    return np.mean(errors)

if __name__ == "__main__":
    # load data
    X_train, digits_train = load_data('./features.train')
    X_test, digits_test = load_data('./features.test')
```

```
    print(f"Training set size: {len(digits_train)}")
    print(f"Test set size: {len(digits_test)}")

    X_train_1v5, y_train_1v5 = one_vs_one(X_train, digits_train, 1, 5)
    X_test_1v5, y_test_1v5 = one_vs_one(X_test, digits_test, 1, 5)

    print(f"1 vs 5 training samples: {len(y_train_1v5)}")
    print(f"1 vs 5 test samples: {len(y_test_1v5)}")

    # P2
    print("\nProblem 2: One v All, C=0.01, Q=2 — Highest E_in")

    C = 0.01
    Q = 2
    results_p2 = {}

    for digit in [0, 2, 4, 6, 8]:
        # binary labels
        y_train = one_vs_all(digits_train, digit)
        y_test = one_vs_all(digits_test, digit)

        # train SVM
        svm = train_svm_poly(X_train, y_train, C, Q)

        # predictions
        y_pred_train = svm.predict(X_train)
        y_pred_test = svm.predict(X_test)

        # compute errors
        e_in = calculate_classification_error(y_train, y_pred_train)
        e_out = calculate_classification_error(y_test, y_pred_test)
        n_sv = get_num_support_vectors(svm)

        results_p2[digit] = {'E_in': e_in, 'E_out': e_out, 'n_sv':
n_sv}
        print(f"  {digit} vs all: E_in = {e_in:.6f}, E_out =
{e_out:.6f}, #SV = {n_sv}")

    # find highest E_in
    best_digit_p2 = max(results_p2, key=lambda d: results_p2[d]
['E_in'])
    print(f"{best_digit_p2} vs all has highest E_in =
{results_p2[best_digit_p2]['E_in']:.6f}")

    # Problem 3
    print("\nProblem 3: One v All, C=0.01, Q=2 — Lowest E_in")

    results_p3 = {}

    for digit in [1, 3, 5, 7, 9]:
```

```
        # create binary labels
        y_train = one_vs_all(digits_train, digit)
        y_test = one_vs_all(digits_test, digit)

        # train SVM
        svm = train_svm_poly(X_train, y_train, C, Q)

        # predictions
        y_pred_train = svm.predict(X_train)
        y_pred_test = svm.predict(X_test)

        # compute errors
        e_in = calculate_classification_error(y_train, y_pred_train)
        e_out = calculate_classification_error(y_test, y_pred_test)
        n_sv = get_num_support_vectors(svm)

        results_p3[digit] = {'E_in': e_in, 'E_out': e_out, 'n_sv':
n_sv}
        print(f"  {digit} vs all: E_in = {e_in:.6f}, E_out =
{e_out:.6f}, #SV = {n_sv}")

    # find lowest E_in
    best_digit_p3 = min(results_p3, key=lambda d: results_p3[d]
['E_in'])
    print(f"{best_digit_p3} vs all has lowest E_in =
{results_p3[best_digit_p3]['E_in']:.6f}")

    # P4
    print("\nProblem 4: Diff in number of support vectors")

    n_sv_highest = results_p2[best_digit_p2]['n_sv']
    n_sv_lowest = results_p3[best_digit_p3]['n_sv']
    diff = abs(n_sv_highest - n_sv_lowest)

    print(f"  Classifier with highest E_in ({best_digit_p2} vs all):
#SV = {n_sv_highest}")
    print(f"  Classifier with lowest E_in ({best_digit_p3} vs all):
#SV = {n_sv_lowest}")
    print(f"  Difference: |{n_sv_highest} - {n_sv_lowest}| = {diff}")

    # P5
    print("\nProblem 5: 1 vs 5 classifier, Q=2, varying C")

    Q = 2
    C_values_p5 = [0.001, 0.01, 0.1, 1]
    results_p5 = {}

    print(f"{'C':<10} {'E_in':<12} {'E_out':<12} {'#SV':<10}")

    for C in C_values_p5:
```

```
        svm = train_svm_poly(X_train_1v5, y_train_1v5, C, Q)

        y_pred_train = svm.predict(X_train_1v5)
        y_pred_test = svm.predict(X_test_1v5)

        e_in = calculate_classification_error(y_train_1v5,
y_pred_train)
        e_out = calculate_classification_error(y_test_1v5,
y_pred_test)
        n_sv = get_num_support_vectors(svm)

        results_p5[C] = {'E_in': e_in, 'E_out': e_out, 'n_sv': n_sv}
        print(f"{C:<10} {e_in:<12.6f} {e_out:<12.6f} {n_sv:<10}")


    sv_counts = [results_p5[C]['n_sv'] for C in C_values_p5]
    e_in_vals = [results_p5[C]['E_in'] for C in C_values_p5]
    e_out_vals = [results_p5[C]['E_out'] for C in C_values_p5]

    # a. #SV strictly decreases as C increases
    a_correct = all(sv_counts[i] > sv_counts[i+1] for i in
range(len(sv_counts)-1))
    print(f"a. #SV strictly decreases: {a_correct}")
    print(f"    SV counts: {sv_counts}")

    # b. #SV strictly increases as C increases
    b_correct = all(sv_counts[i] < sv_counts[i+1] for i in
range(len(sv_counts)-1))
    print(f"b. #SV strictly increases: {b_correct}")

    # c. E_out strictly decreases as C increases
    c_correct = all(e_out_vals[i] > e_out_vals[i+1] for i in
range(len(e_out_vals)-1))
    print(f"c. E_out strictly decreases: {c_correct}")
    print(f"    E_out values: {[f'{v:.6f}' for v in e_out_vals]}")

    # d. Maximum C achieves the lowest E_in
    min_ein = min(e_in_vals)
    d_correct = (results_p5[max(C_values_p5)]['E_in'] == min_ein)
    print(f"d. Max C has lowest E_in: {d_correct}")
    print(f"    E_in values: {[f'{v:.6f}' for v in e_in_vals]}")

    print(f"a,b,c,d: {(a_correct, b_correct, c_correct, d_correct)}
respectively")

    # P6
    print("\nProblem 6: 1 vs 5 classifier, comparing Q=2 with Q=5")

    C_values_p6 = [0.0001, 0.001, 0.01, 1]
    results_p6 = {2: {}, 5: {}}
```

```
    for Q in [2, 5]:
        print(f"\nQ = {Q}:")
        print(f"{'C':<10} {'E_in':<12} {'E_out':<12} {'#SV':<10}")

        for C in C_values_p6:
            svm = train_svm_poly(X_train_1v5, y_train_1v5, C, Q)

            y_pred_train = svm.predict(X_train_1v5)
            y_pred_test = svm.predict(X_test_1v5)

            e_in = calculate_classification_error(y_train_1v5,
y_pred_train)
            e_out = calculate_classification_error(y_test_1v5,
y_pred_test)
            n_sv = get_num_support_vectors(svm)

            results_p6[Q][C] = {'E_in': e_in, 'E_out': e_out, 'n_sv':
n_sv}
            print(f"{C:<10} {e_in:<12.6f} {e_out:<12.6f} {n_sv:<10}")

    # a. C=0.0001: E_in higher at Q=5
    a_correct = results_p6[5][0.0001]['E_in'] > results_p6[2][0.0001]
['E_in']
    print(f"a. C=0.0001: E_in(Q=5)={results_p6[5][0.0001]['E_in']:.6f}
> E_in(Q=2)={results_p6[2][0.0001]['E_in']:.6f}? {a_correct}")

    # b. C=0.001: #SV lower at Q=5
    b_correct = results_p6[5][0.001]['n_sv'] < results_p6[2][0.001]
['n_sv']
    print(f"b. C=0.001: #SV(Q=5)={results_p6[5][0.001]['n_sv']} <
#SV(Q=2)={results_p6[2][0.001]['n_sv']}? {b_correct}")

    # c. C=0.01: E_in higher at Q=5
    c_correct = results_p6[5][0.01]['E_in'] > results_p6[2][0.01]
['E_in']
    print(f"c. C=0.01: E_in(Q=5)={results_p6[5][0.01]['E_in']:.6f} >
E_in(Q=2)={results_p6[2][0.01]['E_in']:.6f}? {c_correct}")

    # d. C=1: E_out lower at Q=5
    d_correct = results_p6[5][1]['E_out'] < results_p6[2][1]['E_out']
    print(f"d. C=1: E_out(Q=5)={results_p6[5][1]['E_out']:.6f} <
E_out(Q=2)={results_p6[2][1]['E_out']:.6f}? {d_correct}")

    print(f"a,b,c,d: {(a_correct, b_correct, c_correct, d_correct)}
respectively")

    # P7 + P8
    print("\nProblem 7 & 8: 1 v 5 classifier, Q=2, 10-fold CV")
```

```
    X_1v5, y_1v5 = one_vs_one(X_train, digits_train, 1, 5)
    print(f"  1 vs 5 training samples: {len(y_1v5)}")

    Q = 2
    C_values = [0.0001, 0.001, 0.01, 0.1, 1]
    n_runs = 100

    # track which C is selected in each run
    selection_counts = {C: 0 for C in C_values}
    selected_ecv_values = []

    for run in range(n_runs):
        # compute E_cv for each C value
        ecv_results = {}
        for C in C_values:
            ecv = cross_validation_error(X_1v5, y_1v5, C, Q,
n_folds=10)
            ecv_results[C] = ecv

        # select C with lowest E_cv (ties broken by smaller C)
        best_C = None
        best_ecv = float('inf')

        for C in C_values:
            if ecv_results[C] < best_ecv:
                best_ecv = ecv_results[C]
                best_C = C

        selection_counts[best_C] += 1
        selected_ecv_values.append(best_ecv)

    # P7
    print("\nProblem 7: Selection frequency across 100 runs")
    for C in C_values:
        print(f"  C = {C}: selected {selection_counts[C]} times")

    most_selected_C = max(selection_counts, key=selection_counts.get)
    print(f"{most_selected_C} is selected most often
({selection_counts[most_selected_C]} times)")

    # P8
    print("\nProblem 8: Average E_cv for overall winning C")

    winning_C = most_selected_C
    avg_ecv = np.mean(selected_ecv_values)
    print(f"  Average E_cv over all 100 runs (selected C's E_cv):
{avg_ecv:.6f}")

    # run CV 100 times with the winning C
    ecv_for_winning = []
```

```
    for run in range(n_runs):
        ecv = cross_validation_error(X_1v5, y_1v5, winning_C, Q,
n_folds=10)
        ecv_for_winning.append(ecv)

    avg_ecv_winning = np.mean(ecv_for_winning)
    print(f"  Average E_cv for C = {winning_C}:
{avg_ecv_winning:.6f}")

    # P9 + P10
    print("Problems 9 & 10: 1 vs 5 classifier with RBF kernel")

    # prepare 1 v 5 test data
    X_test_1v5, y_test_1v5 = one_vs_one(X_test, digits_test, 1, 5)

    C_values_rbf = [0.01, 1, 100, 1e4, 1e6]
    results_rbf = {}

    print(f"\n{'C':<12} {'E_in':<12} {'E_out':<12} {'#SV':<10}")

    for C in C_values_rbf:
        # train SVM with RBF kernel
        # we want gamma=1 to match K(x,y) = exp(-||x-y||^2)
        svm = SVC(kernel='rbf', gamma=1, C=C)
        svm.fit(X_1v5, y_1v5)

        # predictions
        y_pred_train = svm.predict(X_1v5)
        y_pred_test = svm.predict(X_test_1v5)

        # compute errors
        e_in = calculate_classification_error(y_1v5, y_pred_train)
        e_out = calculate_classification_error(y_test_1v5,
y_pred_test)
        n_sv = get_num_support_vectors(svm)

        results_rbf[C] = {'E_in': e_in, 'E_out': e_out, 'n_sv': n_sv}
        print(f"{C:<12} {e_in:<12.6f} {e_out:<12.6f} {n_sv:<10}")

    # P9: find C with lowest E_in
    print("\nProblem 9: Which C results in lowest E_in")
    min_ein_C = min(results_rbf, key=lambda c: results_rbf[c]['E_in'])
    print(f"  Lowest E_in = {results_rbf[min_ein_C]['E_in']:.6f} at C
= {min_ein_C}")

    # P10: find C with lowest E_out
    print("\nProblem 10: Which C results in lowest E_out")
    min_eout_C = min(results_rbf, key=lambda c: results_rbf[c]
['E_out'])
```

```
    print(f"  Lowest E_out = {results_rbf[min_eout_C]['E_out']:.6f} at
C = {min_eout_C}")
```

```
Training set size: 7291
Test set size: 2007
1 vs 5 training samples: 1561
1 vs 5 test samples: 424

Problem 2: One v All, C=0.01, Q=2 — Highest E_in
  0 vs all: E_in = 0.105884, E_out = 0.111609, #SV = 2179
  2 vs all: E_in = 0.100261, E_out = 0.098655, #SV = 1970
  4 vs all: E_in = 0.089425, E_out = 0.099651, #SV = 1856
  6 vs all: E_in = 0.091071, E_out = 0.084704, #SV = 1893
  8 vs all: E_in = 0.074338, E_out = 0.082711, #SV = 1776
0 vs all has highest E_in = 0.105884

Problem 3: One v All, C=0.01, Q=2 — Lowest E_in
  1 vs all: E_in = 0.014401, E_out = 0.021923, #SV = 386
  3 vs all: E_in = 0.090248, E_out = 0.082711, #SV = 1950
  5 vs all: E_in = 0.076258, E_out = 0.079721, #SV = 1585
  7 vs all: E_in = 0.088465, E_out = 0.073244, #SV = 1704
  9 vs all: E_in = 0.088328, E_out = 0.088191, #SV = 1978
1 vs all has lowest E_in = 0.014401

Problem 4: Diff in number of support vectors
  Classifier with highest E_in (0 vs all): #SV = 2179
  Classifier with lowest E_in (1 vs all): #SV = 386
  Difference: |2179 - 386| = 1793

Problem 5: 1 vs 5 classifier, Q=2, varying C
C          E_in         E_out        #SV
0.001      0.004484     0.016509     76
0.01       0.004484     0.018868     34
0.1        0.004484     0.018868     24
1          0.003203     0.018868     24
a. #SV strictly decreases: False
    SV counts: [76, 34, 24, 24]
b. #SV strictly increases: False
c. E_out strictly decreases: False
    E_out values: ['0.016509', '0.018868', '0.018868', '0.018868']
d. Max C has lowest E_in: True
    E_in values: ['0.004484', '0.004484', '0.004484', '0.003203']
a,b,c,d: (False, False, False, np.True_) respectively

Problem 6: 1 vs 5 classifier, comparing Q=2 with Q=5

Q = 2:
C          E_in         E_out        #SV
0.0001     0.008969     0.016509     236
0.001      0.004484     0.016509     76
```

```
0.01       0.004484     0.018868     34
1          0.003203     0.018868     24

Q = 5:
C          E_in         E_out        #SV
0.0001     0.004484     0.018868     26
0.001      0.004484     0.021226     25
0.01       0.003844     0.021226     23
1          0.003203     0.021226     21
a. C=0.0001: E_in(Q=5)=0.004484 > E_in(Q=2)=0.008969? False
b. C=0.001: #SV(Q=5)=25 < #SV(Q=2)=76? True
c. C=0.01: E_in(Q=5)=0.003844 > E_in(Q=2)=0.004484? False
d. C=1: E_out(Q=5)=0.021226 < E_out(Q=2)=0.018868? False
a,b,c,d: (np.False_, True, np.False_, np.False_) respectively

Problem 7 & 8: 1 v 5 classifier, Q=2, 10-fold CV
  1 vs 5 training samples: 1561

Problem 7: Selection frequency across 100 runs
  C = 0.0001: selected 0 times
  C = 0.001: selected 19 times
  C = 0.01: selected 35 times
  C = 0.1: selected 29 times
  C = 1: selected 17 times
0.01 is selected most often (35 times)

Problem 8: Average E_cv for overall winning C
  Average E_cv over all 100 runs (selected C's E_cv): 0.004438
  Average E_cv for C = 0.01: 0.004728
Problems 9 & 10: 1 vs 5 classifier with RBF kernel

C            E_in         E_out        #SV
0.01         0.003844     0.023585     406
1            0.004484     0.021226     31
100          0.003203     0.018868     22
10000.0      0.002562     0.023585     19
1000000.0    0.000641     0.023585     17

Problem 9: Which C results in lowest E_in
  Lowest E_in = 0.000641 at C = 1000000.0

Problem 10: Which C results in lowest E_out
  Lowest E_out = 0.018868 at C = 100
```